



TECHNICAL LEARNING FILE

DEEP-05

Linear Algebra Deep Dive

Geometry and matrix structure for AI

FORMAT	LENGTH	ACCESS
INTENSIVE FILE	50 PAGES	OFFLINE

Edition 2 - original curriculum - editorial review recommended



FILE / ORIENTATION

Purpose

01 FILE GUIDANCE

This optional advanced guide does not gate the core learning path. Apply linear algebra deep dive with explicit assumptions, tests, tradeoffs, and limitations.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



FILE / ORIENTATION

Prerequisites

01 FILE GUIDANCE

Complete the related core track or pass its diagnostic.

NOUS AI ACADEMY

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



FILE / ORIENTATION

Study Method

01 FILE GUIDANCE

For each unit, explain the concept, follow the workflow, reproduce the example, analyze a failure, and complete the applied lab. Record evidence locally.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



UNIT 01 / CONCEPT

Vector Spaces: Concept

01 OBJECTIVE

Explain and apply vector spaces using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Vector Spaces is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Linear Algebra Deep Dive, the terms Vector, Spaces are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should vector spaces be used, and what observable evidence would make that choice defensible? Build a small local example that applies vector spaces, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Vector Spaces in one precise sentence and name one assumption.
2. Create a two-step example of vector spaces and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / WORKFLOW

Vector Spaces: Workflow

01 OBJECTIVE

Explain and apply vector spaces using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Vector Spaces is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to vector spaces.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies vector spaces, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Vector Spaces in one precise sentence and name one assumption.
2. Create a two-step example of vector spaces and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / WORKED EXAMPLE

Vector Spaces: Worked Example

01 OBJECTIVE

Explain and apply vector spaces using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies vector spaces, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns vector spaces into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Vector Spaces in one precise sentence and name one assumption.
2. Create a two-step example of vector spaces and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / FAILURE ANALYSIS

Vector Spaces: Failure Analysis

01 OBJECTIVE

Explain and apply vector spaces using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how vector spaces can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to vector spaces. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Vector Spaces in one precise sentence and name one assumption.
2. Create a two-step example of vector spaces and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / APPLIED LAB

Vector Spaces: Applied Lab

01 OBJECTIVE

Explain and apply vector spaces using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of vector spaces into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies vector spaces, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Vector Spaces in one precise sentence and name one assumption.
2. Create a two-step example of vector spaces and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / CONCEPT

Basis and Coordinates: Concept

01 OBJECTIVE

Explain and apply basis and coordinates using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Basis and Coordinates is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Linear Algebra Deep Dive, the terms Basis, Coordinates are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should basis and coordinates be used, and what observable evidence would make that choice defensible? Build a small local example that applies basis and coordinates, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Basis and Coordinates in one precise sentence and name one assumption.
2. Create a two-step example of basis and coordinates and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / WORKFLOW

Basis and Coordinates: Workflow

01 OBJECTIVE

Explain and apply basis and coordinates using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Basis and Coordinates is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to basis and coordinates.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies basis and coordinates, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Basis and Coordinates in one precise sentence and name one assumption.
2. Create a two-step example of basis and coordinates and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / WORKED EXAMPLE

Basis and Coordinates: Worked Example

01 OBJECTIVE

Explain and apply basis and coordinates using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies basis and coordinates, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns basis and coordinates into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Basis and Coordinates in one precise sentence and name one assumption.
2. Create a two-step example of basis and coordinates and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / FAILURE ANALYSIS

Basis and Coordinates: Failure Analysis

01 OBJECTIVE

Explain and apply basis and coordinates using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how basis and coordinates can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to basis and coordinates. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Basis and Coordinates in one precise sentence and name one assumption.
2. Create a two-step example of basis and coordinates and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / APPLIED LAB

Basis and Coordinates: Applied Lab

01 OBJECTIVE

Explain and apply basis and coordinates using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of basis and coordinates into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies basis and coordinates, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Basis and Coordinates in one precise sentence and name one assumption.
2. Create a two-step example of basis and coordinates and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / CONCEPT

Orthogonality: Concept

01 OBJECTIVE

Explain and apply orthogonality using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Orthogonality is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Linear Algebra Deep Dive, the terms Orthogonality are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should orthogonality be used, and what observable evidence would make that choice defensible? Build a small local example that applies orthogonality, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Orthogonality in one precise sentence and name one assumption.
2. Create a two-step example of orthogonality and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / WORKFLOW

Orthogonality: Workflow

01 OBJECTIVE

Explain and apply orthogonality using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Orthogonality is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to orthogonality.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies orthogonality, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Orthogonality in one precise sentence and name one assumption.
2. Create a two-step example of orthogonality and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / WORKED EXAMPLE

Orthogonality: Worked Example

01 OBJECTIVE

Explain and apply orthogonality using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies orthogonality, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns orthogonality into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Orthogonality in one precise sentence and name one assumption.
2. Create a two-step example of orthogonality and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / FAILURE ANALYSIS

Orthogonality: Failure Analysis

01 OBJECTIVE

Explain and apply orthogonality using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how orthogonality can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to orthogonality. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Orthogonality in one precise sentence and name one assumption.
2. Create a two-step example of orthogonality and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / APPLIED LAB

Orthogonality: Applied Lab

01 OBJECTIVE

Explain and apply orthogonality using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of orthogonality into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies orthogonality, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Orthogonality in one precise sentence and name one assumption.
2. Create a two-step example of orthogonality and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / CONCEPT

Projections: Concept

01 OBJECTIVE

Explain and apply projections using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Projections is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Linear Algebra Deep Dive, the terms Projections are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should projections be used, and what observable evidence would make that choice defensible? Build a small local example that applies projections, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Projections in one precise sentence and name one assumption.
2. Create a two-step example of projections and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / WORKFLOW

Projections: Workflow

01 OBJECTIVE

Explain and apply projections using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Projections is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to projections.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies projections, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Projections in one precise sentence and name one assumption.
2. Create a two-step example of projections and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / WORKED EXAMPLE

Projections: Worked Example

01 OBJECTIVE

Explain and apply projections using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies projections, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns projections into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Projections in one precise sentence and name one assumption.
2. Create a two-step example of projections and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / FAILURE ANALYSIS

Projections: Failure Analysis

01 OBJECTIVE

Explain and apply projections using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how projections can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to projections. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Projections in one precise sentence and name one assumption.
2. Create a two-step example of projections and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / APPLIED LAB

Projections: Applied Lab

01 OBJECTIVE

Explain and apply projections using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of projections into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies projections, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
Model:  $y_{\text{hat}} = w * x + b$   
Loss:  $L = (y_{\text{hat}} - y)^2$   
Gradient:  $dL/dw = 2 * (y_{\text{hat}} - y) * x$   
Update:  $w_{\text{next}} = w - \text{learning\_rate} * dL/dw$ 
```

05 PRACTICE

1. Define Projections in one precise sentence and name one assumption.
2. Create a two-step example of projections and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / CONCEPT

Matrix Rank: Concept

01 OBJECTIVE

Explain and apply matrix rank using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Matrix Rank is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Linear Algebra Deep Dive, the terms Matrix, Rank are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should matrix rank be used, and what observable evidence would make that choice defensible? Build a small local example that applies matrix rank, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Matrix Rank in one precise sentence and name one assumption.
2. Create a two-step example of matrix rank and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / WORKFLOW

Matrix Rank: Workflow

01 OBJECTIVE

Explain and apply matrix rank using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Matrix Rank is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to matrix rank.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies matrix rank, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Matrix Rank in one precise sentence and name one assumption.
2. Create a two-step example of matrix rank and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / WORKED EXAMPLE

Matrix Rank: Worked Example

01 OBJECTIVE

Explain and apply matrix rank using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies matrix rank, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns matrix rank into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Matrix Rank in one precise sentence and name one assumption.
2. Create a two-step example of matrix rank and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / FAILURE ANALYSIS

Matrix Rank: Failure Analysis

01 OBJECTIVE

Explain and apply matrix rank using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how matrix rank can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to matrix rank. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Matrix Rank in one precise sentence and name one assumption.
2. Create a two-step example of matrix rank and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / APPLIED LAB

Matrix Rank: Applied Lab

01 OBJECTIVE

Explain and apply matrix rank using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of matrix rank into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies matrix rank, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Matrix Rank in one precise sentence and name one assumption.
2. Create a two-step example of matrix rank and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / CONCEPT

Eigenvalues: Concept

01 OBJECTIVE

Explain and apply eigenvalues using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Eigenvalues is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Linear Algebra Deep Dive, the terms Eigenvalues are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should eigenvalues be used, and what observable evidence would make that choice defensible? Build a small local example that applies eigenvalues, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Eigenvalues in one precise sentence and name one assumption.
2. Create a two-step example of eigenvalues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / WORKFLOW

Eigenvalues: Workflow

01 OBJECTIVE

Explain and apply eigenvalues using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Eigenvalues is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to eigenvalues.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies eigenvalues, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Eigenvalues in one precise sentence and name one assumption.
2. Create a two-step example of eigenvalues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / WORKED EXAMPLE

Eigenvalues: Worked Example

01 OBJECTIVE

Explain and apply eigenvalues using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies eigenvalues, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns eigenvalues into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Eigenvalues in one precise sentence and name one assumption.
2. Create a two-step example of eigenvalues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / FAILURE ANALYSIS

Eigenvalues: Failure Analysis

01 OBJECTIVE

Explain and apply eigenvalues using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how eigenvalues can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to eigenvalues. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Eigenvalues in one precise sentence and name one assumption.
2. Create a two-step example of eigenvalues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / APPLIED LAB

Eigenvalues: Applied Lab

01 OBJECTIVE

Explain and apply eigenvalues using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of eigenvalues into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies eigenvalues, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Eigenvalues in one precise sentence and name one assumption.
2. Create a two-step example of eigenvalues and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / CONCEPT

Matrix Decompositions: Concept

01 OBJECTIVE

Explain and apply matrix decompositions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Matrix Decompositions is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Linear Algebra Deep Dive, the terms Matrix, Decompositions are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should matrix decompositions be used, and what observable evidence would make that choice defensible? Build a small local example that applies matrix decompositions, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Matrix Decompositions in one precise sentence and name one assumption.
2. Create a two-step example of matrix decompositions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / WORKFLOW

Matrix Decompositions: Workflow

01 OBJECTIVE

Explain and apply matrix decompositions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Matrix Decompositions is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to matrix decompositions.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies matrix decompositions, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Matrix Decompositions in one precise sentence and name one assumption.
2. Create a two-step example of matrix decompositions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / WORKED EXAMPLE

Matrix Decompositions: Worked Example

01 OBJECTIVE

Explain and apply matrix decompositions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies matrix decompositions, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns matrix decompositions into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Matrix Decompositions in one precise sentence and name one assumption.
2. Create a two-step example of matrix decompositions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / FAILURE ANALYSIS

Matrix Decompositions: Failure Analysis

01 OBJECTIVE

Explain and apply matrix decompositions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how matrix decompositions can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to matrix decompositions. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Matrix Decompositions in one precise sentence and name one assumption.
2. Create a two-step example of matrix decompositions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / APPLIED LAB

Matrix Decompositions: Applied Lab

01 OBJECTIVE

Explain and apply matrix decompositions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of matrix decompositions into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies matrix decompositions, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Matrix Decompositions in one precise sentence and name one assumption.
2. Create a two-step example of matrix decompositions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / CONCEPT

Singular Value Decomposition: Concept

01 OBJECTIVE

Explain and apply singular value decomposition using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Singular Value Decomposition is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Linear Algebra Deep Dive, the terms Singular, Value, Decomposition are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should singular value decomposition be used, and what observable evidence would make that choice defensible? Build a small local example that applies singular value decomposition, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Singular Value Decomposition in one precise sentence and name one assumption.
2. Create a two-step example of singular value decomposition and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / WORKFLOW

Singular Value Decomposition: Workflow

01 OBJECTIVE

Explain and apply singular value decomposition using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Singular Value Decomposition is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to singular value decomposition.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies singular value decomposition, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Singular Value Decomposition in one precise sentence and name one assumption.
2. Create a two-step example of singular value decomposition and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / WORKED EXAMPLE

Singular Value Decomposition: Worked Example

01 OBJECTIVE

Explain and apply singular value decomposition using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies singular value decomposition, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns singular value decomposition into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Singular Value Decomposition in one precise sentence and name one assumption.
2. Create a two-step example of singular value decomposition and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / FAILURE ANALYSIS

Singular Value Decomposition: Failure Analysis

01 OBJECTIVE

Explain and apply singular value decomposition using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how singular value decomposition can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to singular value decomposition. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Singular Value Decomposition in one precise sentence and name one assumption.
2. Create a two-step example of singular value decomposition and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / APPLIED LAB

Singular Value Decomposition: Applied Lab

01 OBJECTIVE

Explain and apply singular value decomposition using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of singular value decomposition into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies singular value decomposition, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

Model: $\hat{y} = w * x + b$
Loss: $L = (\hat{y} - y)^2$
Gradient: $dL/dw = 2 * (\hat{y} - y) * x$
Update: $w_{next} = w - learning_rate * dL/dw$

05 PRACTICE

1. Define Singular Value Decomposition in one precise sentence and name one assumption.
2. Create a two-step example of singular value decomposition and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / CONCEPT

Applications in ML: Concept

01 OBJECTIVE

Explain and apply applications in ml using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Applications in ML is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Linear Algebra Deep Dive, the terms Applications are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should applications in ml be used, and what observable evidence would make that choice defensible? Build a small local example that applies applications in ml, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Applications in ML in one precise sentence and name one assumption.
2. Create a two-step example of applications in ml and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / WORKFLOW

Applications in ML: Workflow

01 OBJECTIVE

Explain and apply applications in ml using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Applications in ML is a central part of linear algebra deep dive because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to applications in ml.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies applications in ml, records the result, and tests one failure condition.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Applications in ML in one precise sentence and name one assumption.
2. Create a two-step example of applications in ml and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / WORKED EXAMPLE

Applications in ML: Worked Example

01 OBJECTIVE

Explain and apply applications in ml using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies applications in ml, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns applications in ml into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Applications in ML in one precise sentence and name one assumption.
2. Create a two-step example of applications in ml and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / FAILURE ANALYSIS

Applications in ML: Failure Analysis

01 OBJECTIVE

Explain and apply applications in ml using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how applications in ml can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to applications in ml. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Applications in ML in one precise sentence and name one assumption.
2. Create a two-step example of applications in ml and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / APPLIED LAB

Applications in ML: Applied Lab

01 OBJECTIVE

Explain and apply applications in ml using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of applications in ml into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies applications in ml, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

Model: $y_{\text{hat}} = w * x + b$
Loss: $L = (y_{\text{hat}} - y)^2$
Gradient: $dL/dw = 2 * (y_{\text{hat}} - y) * x$
Update: $w_{\text{next}} = w - \text{learning_rate} * dL/dw$

05 PRACTICE

1. Define Applications in ML in one precise sentence and name one assumption.
2. Create a two-step example of applications in ml and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



FILE / ORIENTATION

Deep-Dive Capstone

01 FILE GUIDANCE

Build a compact, reproducible artifact demonstrating linear algebra deep dive. Include assumptions, a baseline, tests, failure analysis, results, limitations, and instructions for repeating the work offline.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.